

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PROCESOS Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



ESPECIFICACIÓN DE REQUISITOS Y
DOCUMENTACIÓN TÉCNICA

Herramienta de administración para Linux

PROYECTO FINAL

CURSO: PROGRAMACIÓN DE SISTEMAS

DOCENTE: NORMAN PATRICK HARVEY ARCE

INTEGRANTES:

Jimenez Paredes Fabricio Gabriel
Flores Nuñez Rodrigo Francisco

Arequipa-Perú
2026

ESPECIFICACIÓN DE REQUISITOS Y	1
DOCUMENTACIÓN TÉCNICA	1
Control del documento	4
Historial de revisiones	4
1. Introducción	5
1.1 Propósito	5
1.2 Alcance	5
1.3 Público objetivo	5
1.4 Convenciones	5
1.5 Fuentes de información	5
2. Descripción general del sistema	6
2.1 Problema que resuelve	6
2.2 Objetivo general	6
2.3 Objetivos específicos	6
2.4 Perspectiva del producto	6
2.5 Características del usuario	6
2.6 Entorno operativo	6
2.7 Restricciones generales	7
3. Arquitectura y organización del código	7
3.1 Enfoque arquitectónico	7
3.2 Principios de diseño	8
3.3 Módulos principales	9
3.4 Flujo de control	9
3.5 Métricas del código revisado	9
4. Requisitos funcionales	10
4.1 Criterios de aceptación destacados	12
5. Requisitos de interfaces	12
5.1 Interfaz gráfica de usuario	12
5.2 Interfaz de línea de comandos	13
5.3 Interfaces de software	13
5.4 Interfaces de hardware	13
5.5 Interfaces de comunicación	13
6. Diseño técnico por módulo	14
6.1 Administrador de procesos	14
6.2 Explorador y operaciones de archivos	14
6.3 Ejecución de comandos	15
6.4 Respaldos incrementales	16
6.5 Analizador de Bash	17
6.6 Cola de descargas	17
6.7 Utilidades y registros	18
7. Estructuras de datos y API pública	18
7.1 Estructuras principales	18
7.2 Contratos de memoria	18
7.3 API pública resumida	18
8. Requisitos no funcionales	19
8.1 Límites y parámetros de rendimiento	20
9. Escenarios operativos y modelos de interacción	21

9.1 Modelo de casos de uso.....	21
9.2 Escenario A — Controlar un proceso.....	21
9.3 Escenario B — Buscar y copiar una carpeta.....	22
9.4 Escenario C — Ejecutar un comando.....	22
9.5 Escenario D — Crear y restaurar respaldo.....	22
9.6 Escenario E — Analizar script Bash.....	22
10. Instalación, compilación y despliegue.....	23
10.1 Dependencias.....	23
10.2 Compilación.....	23
10.3 Ejecución.....	23
10.4 Productos de construcción.....	23
10.5 Consideraciones de despliegue.....	24
11. Verificación y pruebas.....	24
11.1 Evidencia de revisión.....	24
11.2 Casos de prueba recomendados.....	24
11.3 Pruebas automatizadas futuras.....	25
12. Seguridad, riesgos y limitaciones conocidas.....	25
12.1 Modelo de seguridad actual.....	25
12.2 Riesgos y limitaciones.....	25
12.3 Recomendaciones operativas inmediatas.....	26
13. Mantenimiento y evolución.....	27
13.1 Estrategia de mantenimiento.....	27
13.2 Distribución recomendada del trabajo.....	27
13.3 Convenciones de contribución.....	27
13.4 Hoja de ruta priorizada.....	28
14. Conclusiones técnicas.....	28
15. Referencias.....	29
Apéndice A. Estructura del repositorio.....	30
A.1 Responsabilidad de directorios.....	30
Apéndice B. Matriz de trazabilidad.....	32
Apéndice C. Glosario.....	33

Control del documento

El presente documento consolida la especificación de requisitos, la arquitectura, el diseño técnico, los escenarios operativos, las restricciones, las pruebas y las recomendaciones de mantenimiento del sistema ADMIN. Su estructura adapta el formato SRS proporcionado y lo amplía con secciones de implementación y operación.

Historial de revisiones

Versión	Autor(es)	Descripción	Fecha
0.1	Fabricio G. Jimenez P. y Rodrigo F. Flores N.	Definición inicial de módulos CLI y estructura del proyecto.	11/07/2026
0.8	Fabricio G. Jimenez P. y Rodrigo F. Flores N.	Incorporación de interfaz GTK3, búsqueda global y monitoreo del directorio actual.	18/07/2026
1.0	Fabricio G. Jimenez P. y Rodrigo F. Flores N.	Documentación técnica profesional del estado actual del proyecto.	25/07/2026

1. Introducción

1.1 Propósito

El propósito de este documento es definir de forma verificable qué hace ADMIN, cómo está organizado, cuáles son sus interfaces, qué restricciones técnicas posee y cómo debe compilarse, probarse y mantenerse. El documento sirve como referencia para desarrolladores, revisores académicos, usuarios administradores y futuros responsables de mantenimiento.

1.2 Alcance

ADMIN es una herramienta local para sistemas Linux, desarrollada en C bajo el estándar GNU11. Ofrece una interfaz de línea de comandos y una interfaz gráfica GTK3 sobre una capa núcleo compartida. El sistema integra administración de procesos, exploración y operaciones de archivos, ejecución de comandos, respaldos versionados, análisis estático básico de scripts Bash y una cola de descargas con progreso simulado.

Fuera de alcance — El sistema no sustituye a un administrador de paquetes, no ofrece administración remota, no implementa autenticación propia, no realiza descargas HTTP reales y no garantiza recuperación transaccional ante interrupciones durante una copia o respaldo.

1.3 Público objetivo

- Usuarios de Linux con conocimientos básicos de archivos, procesos y terminal.
- Desarrolladores en C responsables de ampliar o corregir los módulos.
- Docentes y revisores que necesiten evaluar requisitos, arquitectura y trazabilidad.
- Administradores que ejecuten la herramienta con los permisos de su propia cuenta.

1.4 Convenciones

Convención	Significado
RF-XXX-NN	Requisito funcional identificado por módulo.
RNF-XXX-NN	Requisito no funcional o atributo de calidad.
CLI	Interfaz de línea de comandos.
GUI	Interfaz gráfica basada en GTK3.
Implementado / Parcial / Planificado	Estado del requisito respecto del código revisado.
Debe	Condición obligatoria para considerar satisfecho un requisito.

1.5 Fuentes de información

La especificación se elaboró a partir del código fuente y archivos de construcción del proyecto, del formato SRS suministrado como referencia y de documentación primaria de ISO/IEC/IEEE, GTK/GLib, GNU, The Open Group y el kernel de Linux. Las referencias completas aparecen en la sección 15.

2. Descripción general del sistema

2.1 Problema que resuelve

Las tareas habituales de administración en Linux suelen distribuirse entre múltiples comandos y utilidades. ADMIN reúne operaciones frecuentes en una aplicación educativa única, permitiendo observar procesos, manipular archivos, ejecutar comandos, generar respaldos y revisar scripts sin cambiar de herramienta.

2.2 Objetivo general

Desarrollar una herramienta modular para Linux que facilite actividades básicas de administración del sistema mediante una interfaz CLI y una interfaz gráfica, reutilizando una misma capa de lógica para evitar duplicación de código.

2.3 Objetivos específicos

1. Consultar información de procesos y enviar señales de control dentro de los permisos del usuario.
2. Gestionar archivos y carpetas, incluyendo búsqueda recursiva local o global y actualización automática del directorio visualizado.
3. Ejecutar comandos del sistema y presentar por separado la salida estándar, la salida de error y el código de terminación.
4. Crear respaldos versionados y restaurar el contenido de una versión seleccionada.
5. Analizar patrones básicos de scripts Bash y presentar un resumen estructurado.
6. Mantener una arquitectura modular, compilable con GNU Make y ampliable por módulos.

2.4 Perspectiva del producto

El programa es una aplicación de escritorio local y autónoma. No depende de una base de datos ni de un servidor. La información de procesos se obtiene desde /proc; las operaciones de archivos se realizan sobre el sistema de archivos local; el historial y los registros se guardan en archivos de texto relativos al directorio de ejecución.

2.5 Características del usuario

Perfil	Conocimientos esperados	Acciones principales
Usuario básico	Rutas, carpetas, archivos y uso elemental de Linux.	Navegar, buscar, consultar estadísticas y crear respaldos.
Usuario administrador	Procesos, señales, permisos y consecuencias de ejecutar comandos.	Finalizar o suspender procesos, ejecutar comandos y restaurar datos.
Desarrollador	C, POSIX, GTK3, GLib/GIO y GNU Make.	Mantener, probar y extender el sistema.

2.6 Entorno operativo

Elemento	Especificación
Sistema operativo	Linux. El README indica Ubuntu 24.04 como entorno probado por el equipo.
Lenguaje	C con opciones de compilación -std=gnu11.

Elemento	Especificación
Interfaz gráfica	GTK 3.24 mediante libgtk-3-dev y pkg-config.
Compilación	gcc y GNU Make.
Servicios del sistema	/proc, señales POSIX, fork/exec, pipes, llamadas de archivos y GLib/GIO.
Persistencia	Archivos de log, manifiesto .ultimo_respaldo y carpetas de versiones.

2.7 Restricciones generales

- La administración de procesos y archivos está limitada por los permisos del usuario que ejecuta la aplicación.
- La GUI requiere bibliotecas de desarrollo GTK3 durante la compilación.
- El proyecto usa rutas y mecanismos propios de Linux; no es portable directamente a Windows o macOS.
- La cola de descargas actual simula progreso y no transfiere datos por red.
- La búsqueda global omite sistemas virtuales y limita resultados para reducir riesgo de bloqueo.

3. Arquitectura y organización del código

3.1 Enfoque arquitectónico

La arquitectura separa presentación y lógica. main.c implementa menús de texto, mientras que main_gui.c y src/gui/*.c construyen la aplicación GTK3. Ambos frentes llaman a las funciones públicas declaradas en include/*.h y ejecutadas por los módulos núcleo.

Arquitectura lógica del sistema ADMIN

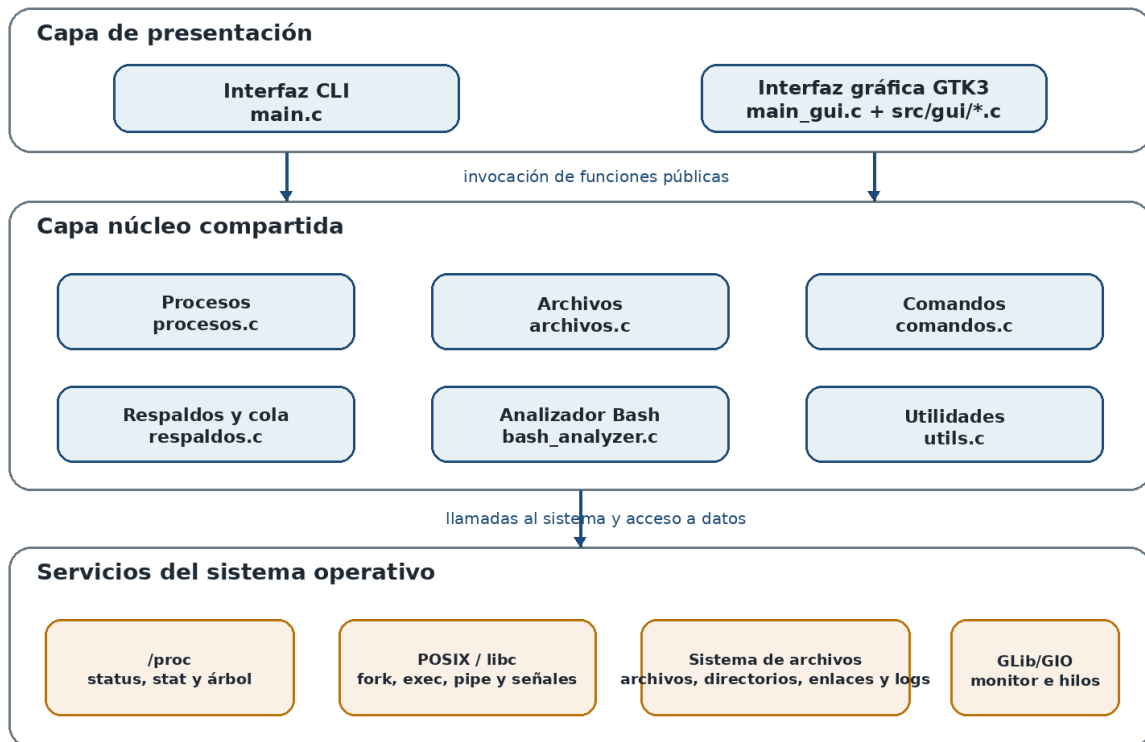


Figura 1. Arquitectura lógica por capas del sistema ADMIN.

3.2 Principios de diseño

Principio	Aplicación en el proyecto
Separación de responsabilidades	La GUI contiene widgets y callbacks; los módulos <code>src/*.c</code> contienen la lógica del sistema.
Reutilización	Las interfaces CLI y GUI comparten <code>procesos.c</code> , <code>archivos.c</code> , <code>comandos.c</code> , <code>respaldos.c</code> , <code>bash_analyzer.c</code> y <code>utils.c</code> .
Encapsulamiento	Las cabeceras <code>include/*.h</code> exponen estructuras y funciones públicas; las funciones auxiliares son <code>static</code> .
Datos estructurados	La GUI recibe <code>ListaProcesos</code> , <code>ListaArchivos</code> , <code>ResultadoComando</code> y <code>ResultadoAnálisisBash</code> en vez de analizar texto impreso.
Manejo explícito de memoria	Las funciones que reservan memoria indican que el llamador debe liberarla con <code>free()</code> o <code>g_free()</code> .
Construcción reproducible	El Makefile define objetivos independientes para CLI, GUI, ejecución y limpieza.

3.3 Módulos principales

Módulo	Responsabilidad	Dependencias principales
procesos.c	Consulta de procesos, CPU/memoria, señales y árbol.	/proc, signal.h, unistd.h
archivos.c	Listado, copia recursiva, movimiento, eliminación, creación, búsqueda y estadísticas.	dirent.h, stat, open/read/write, symlink
comandos.c	Ejecución de comandos y captura de salidas.	fork, exec, pipe, dup2, waitpid
respaldos.c	Versionado incremental, restauración y cola simulada.	archivos.c, stat, time, directorios
bash_analyzer.c	Conteo de variables y estructuras Bash.	Lectura de texto y análisis por patrones
utils.c	Formato, marcas de tiempo y registros.	stdio, time
src/gui/*.c	Presentación por pestañas y callbacks.	GTK3, GLib/GIO

3.4 Flujo de control

En la CLI, el usuario selecciona un menú y main.c invoca directamente el módulo correspondiente. En la GUI, GtkApplication crea una ventana con GtkNotebook; cada pestaña registra señales de botones y entradas. Los callbacks obtienen datos del núcleo, actualizan GtkListStore o GtkTextBuffer y muestran diálogos de estado.

3.5 Métricas del código revisado

Métrica	Valor	Observación
Archivos C y cabeceras	20	14 archivos .c y 6 archivos .h.
Líneas en fuentes .c	2 931	Incluye GUI y lógica núcleo.
Líneas en cabeceras .h	176	Interfaces públicas y estructuras.
Total aproximado de código	3 107 líneas	No incluye README, documentación ni Makefile.
Ejecutables previstos	2	admin (CLI) y admin-gui (GTK3).
Estándar / advertencias	GNU11, -Wall -Wextra	Compilación con símbolos de depuración (-g).

4. Requisitos funcionales

Los requisitos siguientes describen el comportamiento observable del estado actual del proyecto. La prioridad Alta identifica funciones centrales o con impacto destructivo; Media identifica funciones complementarias.

ID	Módulo	Descripción verificable	Prioridad	Estado
RF-PRO-01	Procesos	Listar procesos activos con PID, PPID, nombre, estado y memoria RSS.	Alta	Implementado
RF-PRO-02	Procesos	Buscar procesos mediante coincidencia por nombre.	Media	Implementado
RF-PRO-03	Procesos	Obtener CPU estimada y memoria del PID seleccionado.	Alta	Implementado
RF-PRO-04	Procesos	Enviar SIGTERM para solicitar la finalización de un proceso.	Alta	Implementado
RF-PRO-05	Procesos	Suspender y reanudar procesos mediante SIGSTOP y SIGCONT.	Alta	Implementado
RF-PRO-06	Procesos	Construir una vista jerárquica de procesos desde PID 1.	Media	Implementado
RF-ARC-01	Archivos	Navegar al inicio, raíz, directorio padre o ruta escrita; abrir carpetas con doble clic.	Alta	Implementado
RF-ARC-02	Archivos	Actualizar la vista cuando cambie el directorio actualmente monitoreado.	Alta	Implementado
RF-ARC-03	Archivos	Mostrar nombre, tipo, tamaño y ubicación, con capacidad de ordenamiento.	Alta	Implementado
RF-ARC-04	Archivos	Crear un archivo vacío o una carpeta en el directorio actual.	Media	Implementado en GUI
RF-ARC-05	Archivos	Copiar archivos, enlaces simbólicos y carpetas completas de forma recursiva.	Alta	Implementado
RF-ARC-06	Archivos	Impedir copiar una carpeta dentro de sí misma y evitar mezclar con una carpeta destino existente.	Alta	Implementado
RF-ARC-07	Archivos	Mover elementos con rename(); para archivos regulares entre sistemas, usar copia y eliminación.	Alta	Parcial
RF-ARC-08	Archivos	Eliminar archivos o directorios vacíos; la GUI debe pedir confirmación.	Alta	Implementado

ID	Módulo	Descripción verificable	Prioridad	Estado
RF-ARC-09	Archivos	Buscar recursivamente en la carpeta actual o desde / sin bloquear la GUI.	Alta	Implementado
RF-ARC-10	Archivos	Mostrar tamaño, tipo, permisos y fecha de modificación.	Media	Implementado
RF-CMD-01	Comandos	Ejecutar una cadena mediante /bin/sh -c.	Alta	Implementado
RF-CMD-02	Comandos	Capturar stdout, stderr y código de salida por separado.	Alta	Implementado
RF-CMD-03	Comandos	Registrar cada comando con fecha y hora.	Media	Implementado
RF-CMD-04	Comandos	Consultar y limpiar el historial persistente.	Media	Implementado
RF-RES-01	Respaldos	Crear un respaldo completo inicial y versiones incrementales posteriores basadas en mtime.	Alta	Implementado
RF-RES-02	Respaldos	Nombrar las versiones con una marca de tiempo y registrar la fecha del último respaldo.	Media	Implementado
RF-RES-03	Respaldos	Listar versiones existentes.	Media	Implementado
RF-RES-04	Respaldos	Restaurar los archivos de una versión seleccionada en una ruta destino.	Alta	Parcial
RF-BAS-01	Bash	Detectar asignaciones de variables y evitar duplicados en el conteo.	Media	Implementado
RF-BAS-02	Bash	Contar for, while, until, if y definiciones de funciones; generar detalle y resumen.	Media	Implementado
RF-COL-01	Cola	Agregar hasta 64 URL o rutas a una cola en memoria.	Media	Implementado
RF-COL-02	Cola	Mostrar los elementos pendientes.	Media	Implementado
RF-COL-03	Cola	Procesar la cola mostrando progreso y registrando finalización.	Media	Simulado
RF-UI-01	Interfaces	Ofrecer CLI y GUI sobre una misma capa núcleo.	Alta	Implementado

ID	Módulo	Descripción verificable	Prioridad	Estado
RF-UI-02	Interfaces	Organizar la GUI en pestañas de Procesos, Archivos, Comandos y Respaldos/Bash/Descargas.	Alta	Implementado

4.1 Criterios de aceptación destacados

Requisito	Criterio de aceptación
RF-PRO-01	Al actualizar, la tabla contiene procesos accesibles desde /proc con columnas PID, PPID, nombre, estado y memoria.
RF-ARC-02	Al crear, borrar o renombrar un elemento en la carpeta visualizada, la lista se recarga sin pulsar "Listar / actualizar".
RF-ARC-05	La copia de una carpeta reproduce subcarpetas, archivos regulares y enlaces simbólicos sin crear el destino dentro del origen.
RF-ARC-09	La ventana permanece interactiva durante la búsqueda; el resultado se limita a 5000 entradas y excluye /proc, /sys, /dev y /run.
RF-CMD-02	El resultado presenta dos textos separados y el código de salida devuelto por waitpid().
RF-RES-01	La primera ejecución copia todos los archivos; las siguientes copian solo archivos cuyo mtime sea posterior al manifiesto.
RF-BAS-02	El resumen coincide con las estructuras detectadas en el script de prueba incluido.
RF-COL-03	La barra o terminal avanza hasta 100 %, vacía la cola y crea un registro; no se exige transferencia real.

5. Requisitos de interfaces

5.1 Interfaz gráfica de usuario

La GUI utiliza GtkApplication para administrar el ciclo de vida de la aplicación y GtkApplicationWindow como ventana principal. Un GtkNotebook organiza las funciones en cuatro pestañas. Las tablas se implementan con GtkTreeView y GtkListStore; los resultados textuales se muestran en GtkTextView o diálogos desplazables.

Pestaña	Controles principales	Salida
Procesos	Buscar, actualizar, ver CPU/memoria, finalizar, suspender, reanudar y ver árbol.	Tabla de procesos y diálogos.

Pestaña	Controles principales	Salida
Archivos	Inicio, raíz, subir, ruta, búsqueda global, crear, copiar, mover, eliminar y estadísticas.	Tabla de archivos con ubicación y etiqueta de estado.
Comandos	Entrada de comando, ejecutar, historial, limpiar historial y pantalla.	Texto coloreado para stdout, stderr y estado.
RespalDOS / Bash / Descargas	Rutas de origen/destino, versiones, restauración, script, cola y barra de progreso.	Mensajes, resumen Bash, lista y progreso.

5.2 Interfaz de línea de comandos

La CLI presenta un menú principal y submenús numéricos. Las entradas se leen con `fgets()`, se elimina el salto de línea y las opciones numéricas se convierten con `atoi()`. Los resultados emplean colores ANSI para distinguir títulos, éxito, información y errores.

5.3 Interfaces de software

Interfaz	Uso en ADMIN
/proc	Lectura de status, stat y datos globales de CPU.
POSIX / libc	Procesos, señales, pipes, archivos, directorios y espera de hijos.
GTK3	Ventana, pestañas, tablas, entradas, diálogos, texto y barra de progreso.
Glib/GIO	Monitoreo de directorios, GThread, <code>g_idle_add</code> y utilidades de rutas/cadenas.
GNU Make / pkg-config	Compilación incremental y resolución de flags GTK3.
/bin/sh	Interpretación de los comandos introducidos por el usuario.

5.4 Interfaces de hardware

No se requiere hardware especializado. La aplicación utiliza CPU para recorridos y muestreo, memoria para listas y buffers, y almacenamiento para operaciones, respaldos e historial. Las acciones solo alcanzan recursos visibles y permitidos por el kernel para el usuario actual.

5.5 Interfaces de comunicación

La versión actual no abre sockets ni implementa protocolos de red. La “cola de descargas” mantiene cadenas en memoria y simula su avance. Una versión real deberá incorporar HTTP seguro, validación de URL, cancelación, límites de tamaño y verificación de integridad.

6. Diseño técnico por módulo

6.1 Administrador de procesos

procesos_obtener_info() abre /proc/[pid]/status y extrae Name, State, PPid y VmRSS. La estimación de CPU toma dos muestras separadas por 150 ms: suma utime y stime del proceso, calcula la diferencia respecto del total de /proc/stat y multiplica por la cantidad de CPU en línea.

Función pública	Descripción técnica
procesos_obtener_lista() ()	Recorre entradas numéricas de /proc y construye un arreglo dinámico de InfoProceso.
procesos_buscar_lista()	Filtra la lista completa mediante strstr() sobre el nombre.
procesos_calcular_cpu_pct() _pct()	Muestrea ticks del proceso y del sistema; devuelve porcentaje estimado.
procesos_finalizar()	Envía SIGTERM con kill().
procesos_suspender()/r eanudar()	Envían SIGSTOP y SIGCONT.
procesos_arbol_texto()	Guarda hasta 2048 pares PID/PPID y construye texto recursivo desde PID 1.

6.2 Explorador y operaciones de archivos

archivos.c emplea lstat() para distinguir archivos, directorios y enlaces simbólicos. El listado ordena primero directorios y luego nombres sin distinguir mayúsculas. La copia recursiva usa open/read/write para archivos regulares, readlink/symlink para enlaces y mkdir para directorios.

Aspecto	Implementación
Copia de archivos	Buffer de 64 KiB, escritura parcial controlada y permisos derivados del origen.
Copia de carpetas	Recorrido recursivo; se rechaza un destino dentro del origen y no se mezclan directorios existentes.
Movimiento	rename(); si falla y el origen es archivo regular, copia y elimina.
Eliminación	remove() para archivos y rmdir() para carpetas vacías.
Búsqueda	Recorrido recursivo por subcadena; máximo 5000 resultados; exclusión de rutas virtuales.
Monitoreo GUI	GFileMonitor observa el directorio actual y programa una recarga con retardo para agrupar eventos.
Concurrencia	La búsqueda global se ejecuta en GThread y entrega resultados al hilo GTK mediante g_idle_add().

Secuencia: búsqueda global de archivos

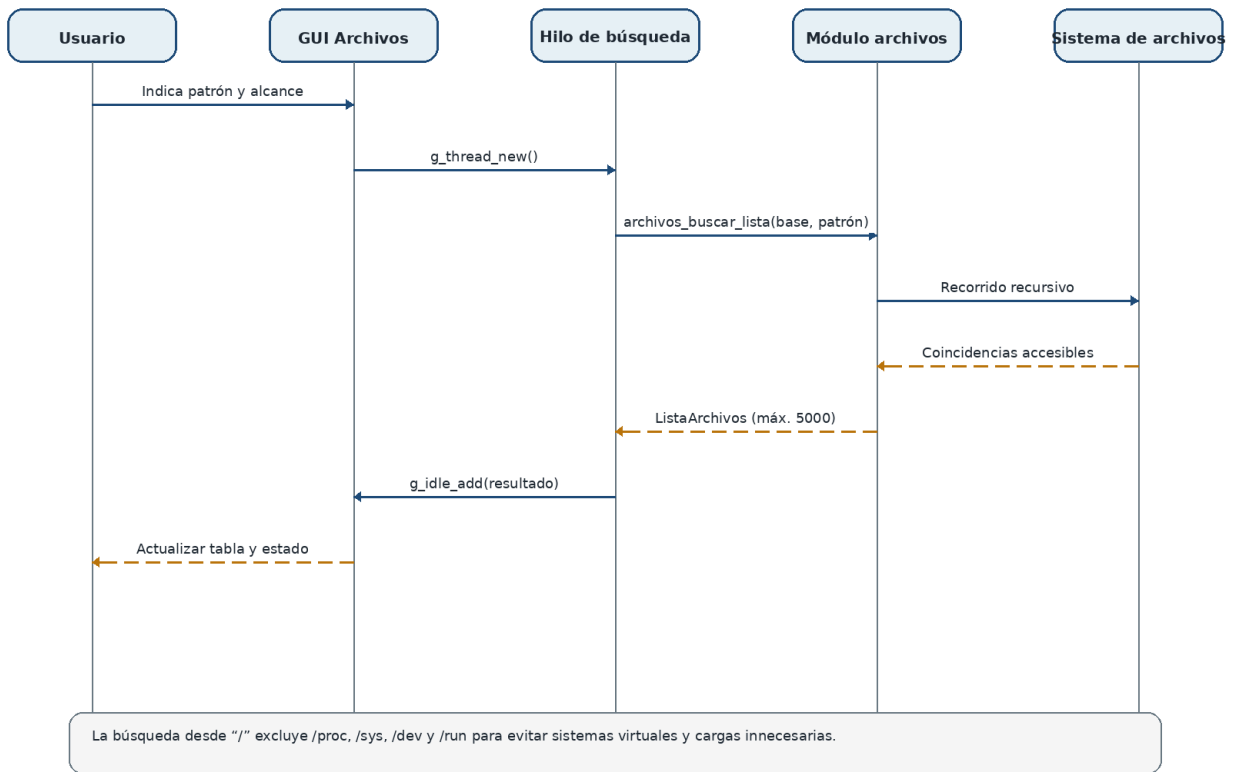


Figura 2. Secuencia de la búsqueda global sin bloqueo de la interfaz.

6.3 Ejecución de comandos

comandos_ejecutar_capturar() crea dos pipes, ejecuta fork(), redirige stdout y stderr del proceso hijo con dup2() y reemplaza el hijo por /bin/sh -c mediante execl(). El padre lee las salidas en buffers dinámicos, espera con waitpid() y registra el comando con una marca temporal.

Secuencia: ejecución de un comando Linux

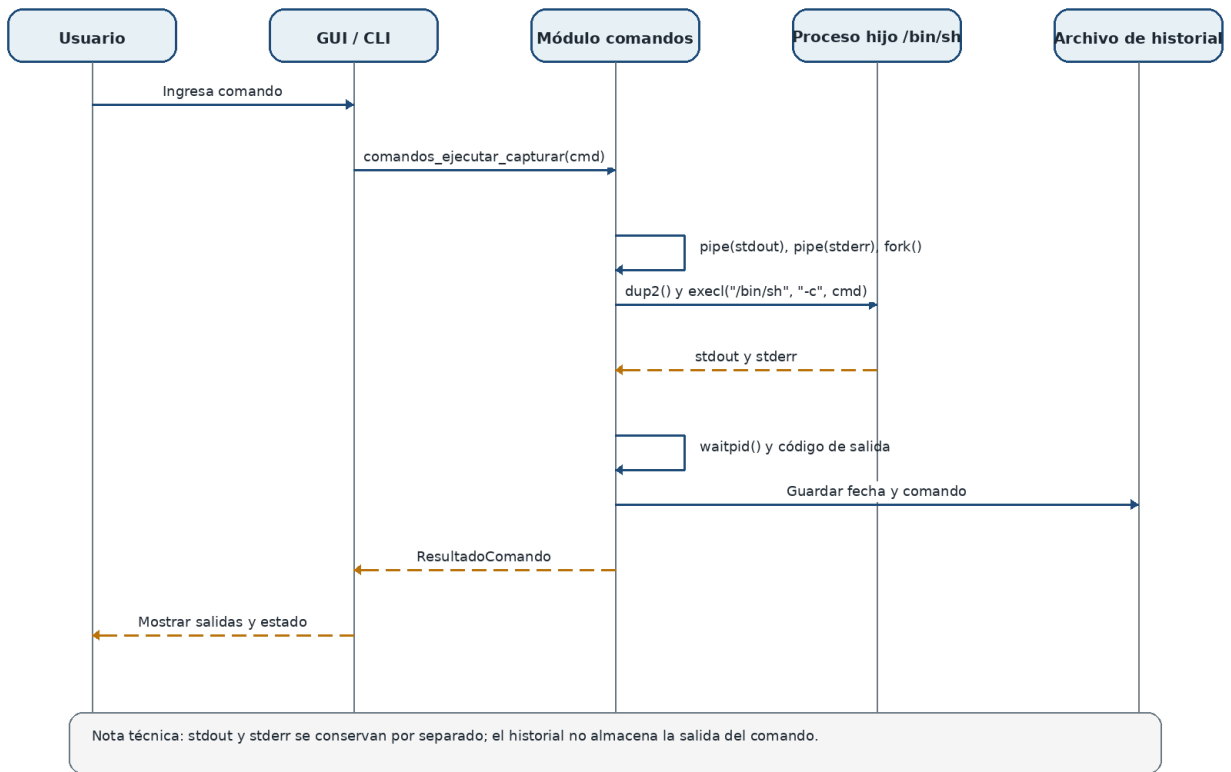


Figura 3. Secuencia de ejecución y captura de un comando.

6.4 Respallos incrementales

El módulo almacena en destino_base/.ultimo_respaldo la fecha Unix del último respaldo. Cada nueva versión usa una carpeta con marca de tiempo. En la primera ejecución se copian todos los archivos; luego se copian los archivos cuyo st_mtime sea posterior al valor guardado.

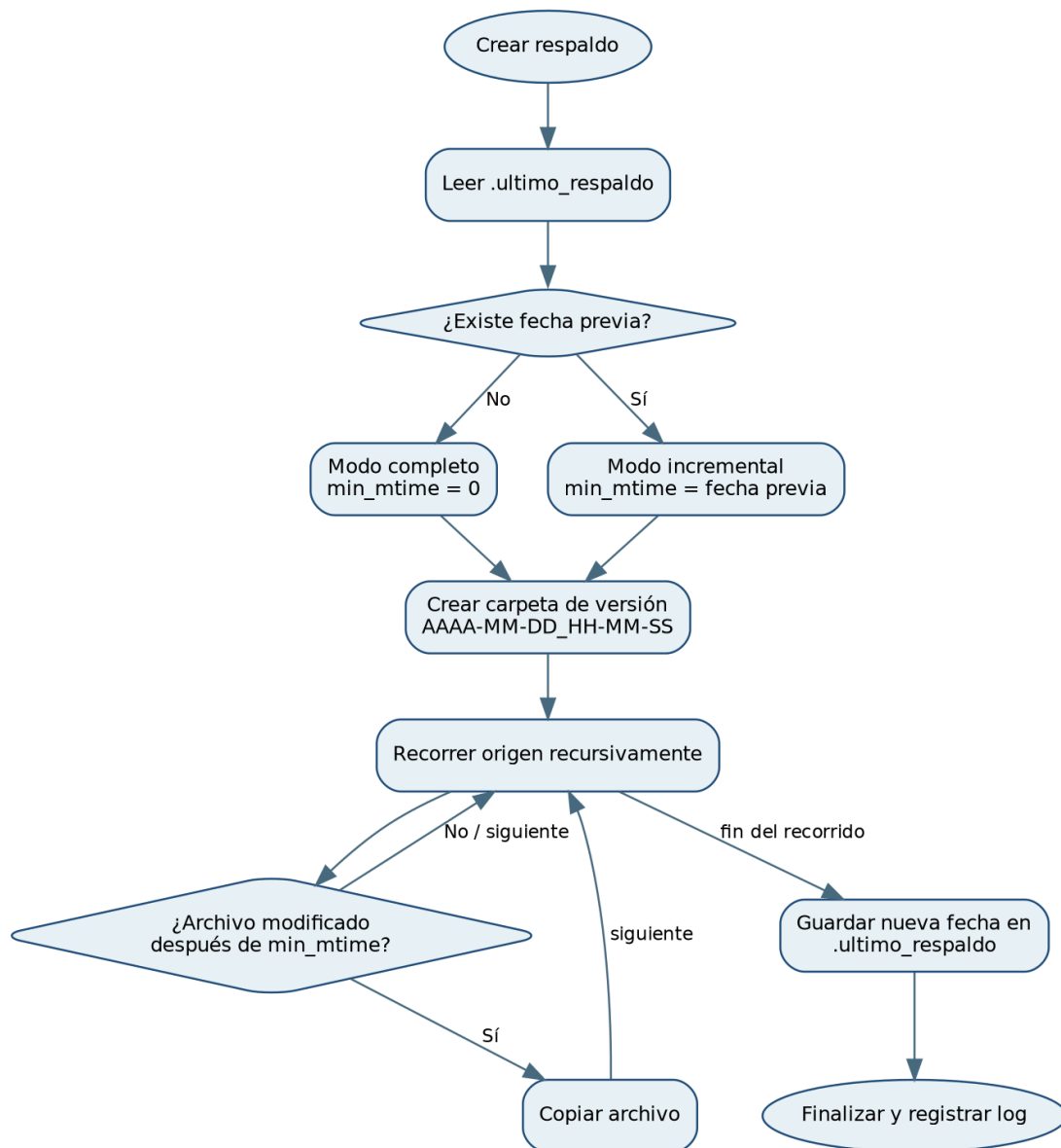


Figura 4. Flujo actual de creación de respaldos.

6.5 Analizador de Bash

El analizador realiza un reconocimiento léxico sencillo por línea. Detecta asignaciones con un identificador válido antes del signo igual, omite comentarios y espacios iniciales, mantiene una lista de variables únicas y cuenta estructuras por prefijos o patrones. No construye un árbol sintáctico y no interpreta expansiones complejas.

6.6 Cola de descargas

La cola usa un arreglo global de 64 cadenas de hasta 511 caracteres. El procesamiento recorre cada elemento y genera porcentajes periódicos mediante `usleep()`. La GUI recibe un callback para actualizar `GtkProgressBar` y procesa eventos GTK pendientes. Esta función debe considerarse una demostración de cola y progreso, no un cliente de descarga.

6.7 Utilidades y registros

utils_timestamp() produce marcas de tiempo utilizadas para historial y versiones. utils_log() agrega eventos a archivos dentro de logs/. El directorio se crea al iniciar la CLI o la GUI y también mediante el objetivo dirs del Makefile.

7. Estructuras de datos y API pública

7.1 Estructuras principales

Estructura	Campos relevantes	Responsabilidad
InfoProceso	pid, ppid, nombre, estado, mem_kb, cpu_pct	Representar un proceso accesible.
ListaProcesos	InfoProceso *items, total	Transferir listas dinámicas a la GUI.
ItemArchivo	nombre, ruta, es_directorio, tamano	Representar un resultado de listado o búsqueda.
ListaArchivos	ItemArchivo *items, total, truncada	Lista dinámica y señal de límite alcanzado.
ResultadoComando	salida_estandar, salida_error, codigo_salida	Resultado completo de una ejecución.
ResultadoAnalisisBash	contadores y detalle	Resumen y explicación del análisis.
ListaVersiones	64 nombres, total	Versiones de respaldo para el combo GUI.
ColaSnapshot	64 elementos, total	Copia visible de la cola en memoria.

7.2 Contratos de memoria

Regla — Las funciones que devuelven char* o arreglos dinámicos reservados con malloc deben liberarse con free(). Las cadenas devueltas por gui_pedir_texto() se reservan con g_strdup() y deben liberarse con g_free(). Las listas tienen funciones explícitas de liberación.

7.3 API pública resumida

Cabecera	Operaciones principales
procesos.h	procesos_obtener_lista, procesos_buscar_lista, procesos_calcular_cpu_pct, procesos_finalizar, procesos_suspender, procesos_reanudar, procesos_arbol_texto
archivos.h	archivos_obtener_lista, archivos_buscar_lista, archivos_copiar, archivos_mover, archivos_eliminar, archivos_crear_directorio, archivos_crear_archivo, archivos_estadisticas_texto

Cabecera	Operaciones principales
comandos.h	comandos_ejecutar_capturar, comandos_obtener_historial_texto, comandos_limpiar_historial
respaldos.h	respaldos_crear_incremental, respaldos_obtener_versiones, respaldos_restaurar, respaldos_cola_agregar, respaldos_cola_obtener, respaldos_cola_procesar_cb
bash_analyzer.h	bash_analizar_script_datos
gui_common.h	diálogos reutilizables y fábricas de las cuatro pestañas

8. Requisitos no funcionales

Los atributos se organizan con base en modelos de calidad de producto de software. Los valores propuestos son verificables y distinguen el estado actual de las mejoras recomendadas.

ID	Atributo	Requisito	Estado
RNF-USA-01	Usabilidad	La GUI debe agrupar funciones por pestañas y mostrar errores comprensibles sin cerrar la aplicación.	Implementado
RNF-REN-01	Eficiencia	La búsqueda de archivos no debe ejecutarse en el hilo principal GTK.	Implementado
RNF-REN-02	Eficiencia	La búsqueda debe detener la recolección al alcanzar 5000 resultados.	Implementado
RNF-FIA-01	Fiabilidad	Los descriptores, archivos y directorios abiertos deben cerrarse en rutas normales y de error.	Mayormente implementado
RNF-SEG-01	Seguridad	La aplicación debe ejecutarse con el menor privilegio posible.	Recomendación operativa
RNF-SEG-02	Seguridad	Las operaciones destructivas en GUI deben requerir confirmación.	Parcial: archivos sí; procesos no
RNF-MAN-01	Mantenibilidad	Cada dominio debe conservar su cabecera y módulo independiente.	Implementado
RNF-MAN-02	Mantenibilidad	La compilación debe usar -Wall -Wextra y no introducir advertencias nuevas.	Definido en Makefile

ID	Atributo	Requisito	Estado
RNF-POR-01	Portabilidad	El sistema debe compilar en distribuciones Linux con GCC, Make y GTK3.	Objetivo actual
RNF-COM-01	Compatibilidad	CLI y GUI deben usar la misma API núcleo y producir resultados equivalentes.	Implementado
RNF-OBS-01	Observabilidad	Comandos, archivos, respaldos y cola deben registrar eventos relevantes.	Parcial
RNF-REC-01	Recuperabilidad	La restauración debe reconstruir un estado completo y verificable.	No satisfecho completamente

8.1 Límites y parámetros de rendimiento

Parámetro	Valor actual	Implicación
Muestreo de CPU	150 ms	Cada consulta de un PID introduce aproximadamente ese tiempo de observación.
Resultados de búsqueda	5000	Evita crecimiento ilimitado de la tabla y memoria.
Procesos en árbol	2048	Procesos adicionales no aparecen en la representación jerárquica.
Cola	64 elementos	Los elementos viven únicamente durante la ejecución.
PATH_MAX	4096 cuando no está definido	Rutas más largas se rechazan o truncan según la operación.
Buffer de copia	64 KiB	Compromiso razonable entre memoria y número de llamadas.
Buffers de comandos	Crecimiento dinámico desde 4096 bytes	La memoria aumenta con el volumen de salida.

9. Escenarios operativos y modelos de interacción

9.1 Modelo de casos de uso

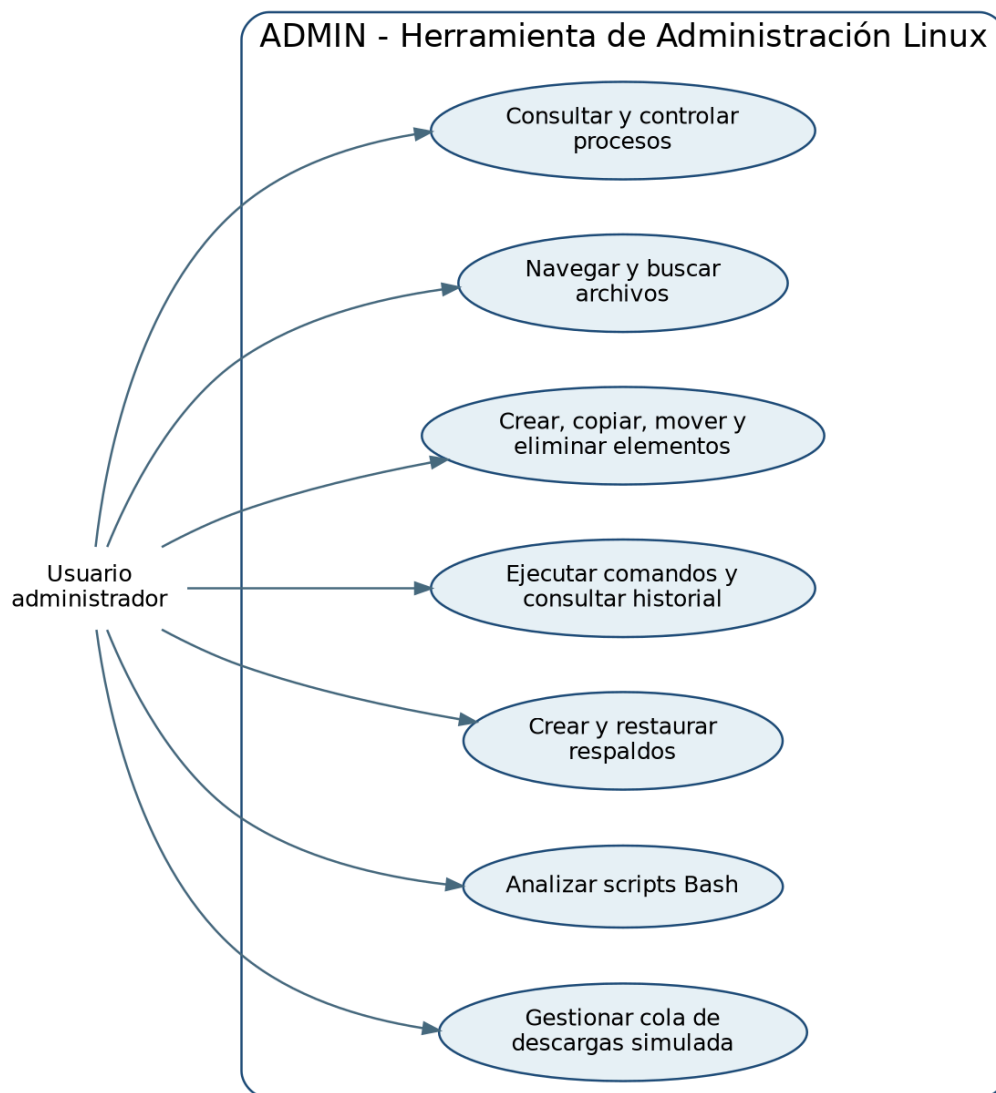


Figura 5. Casos de uso principales del usuario administrador.

9.2 Escenario A — Controlar un proceso

Elemento	Descripción
Precondición	La aplicación se ejecuta en Linux y el proceso es visible en /proc.
Flujo principal	Actualizar lista → seleccionar PID → consultar CPU/memoria → elegir suspender, reanudar o finalizar.
Resultado	Se envía la señal y se actualiza la tabla si el kernel autoriza la operación.
Alternativas	El proceso terminó antes de la acción o el usuario carece de permisos; se muestra un error.

9.3 Escenario B — Buscar y copiar una carpeta

Elemento	Descripción
Precondición	La carpeta origen es legible y el destino no existe o es una carpeta contenedora válida.
Flujo principal	Elegir alcance global → buscar por nombre → seleccionar carpeta → Copiar → escribir destino.
Resultado	Se crea una copia recursiva con archivos, subcarpetas y enlaces simbólicos.
Alternativas	El destino está dentro del origen, ya existe o no hay permisos; la operación se rechaza con detalle.

9.4 Escenario C — Ejecutar un comando

Elemento	Descripción
Precondición	El comando existe o puede ser interpretado por /bin/sh; el usuario comprende sus efectos.
Flujo principal	Escribir comando → ejecutar → revisar stdout, stderr y código de salida.
Resultado	El comando queda registrado en logs/historial_comandos.log.
Alternativas	fork, pipe o exec fallan; se reporta salida de error o código 127.

9.5 Escenario D — Crear y restaurar respaldo

Elemento	Descripción
Precondición	Origen legible y destino de respaldos escribible.
Flujo principal	Indicar origen/destino → crear versión → listar versiones → seleccionar y restaurar.
Resultado	Se copian los archivos de la versión seleccionada en la ruta de restauración.
Advertencia	Una versión incremental aislada no contiene necesariamente todos los archivos del estado original.

9.6 Escenario E — Analizar script Bash

Elemento	Descripción
Precondición	Archivo de texto legible, preferentemente con extensión .sh.
Flujo principal	Indicar ruta → Analizar → revisar hallazgos y resumen.

Elemento	Descripción
Resultado	Se muestran variables únicas y conteos de estructuras reconocidas.
Limitación	El análisis es heurístico; no valida sintaxis Bash completa ni expansiones complejas.

10. Instalación, compilación y despliegue

10.1 Dependencias

```
sudo apt update
sudo apt install -y build-essential pkg-config libgtk-3-dev
```

10.2 Compilación

```
cd admin_proyecto
make clean
make          # compila CLI y GUI

# Objetivos alternativos
make cli
make gui
```

10.3 Ejecución

```
./admin          # interfaz de terminal
./admin-gui      # interfaz gráfica

# O mediante Make
make run
make run-gui
```

10.4 Productos de construcción

Producto	Origen	Descripción
admin	CORE_OBJS + main.o	Ejecutable de línea de comandos.
admin-gui	CORE_OBJS + main_gui.o + src/gui/*.o	Ejecutable GTK3.
obj/	Reglas de compilación	Archivos objeto; incluye obj/gui/.
logs/	Creado por dirs o al iniciar	Historial y registros.

Producto	Origen	Descripción
respaldos_data/	Creado por dirs	Directorio reservado para respaldos.

10.5 Consideraciones de despliegue

- Ejecutar desde un directorio en el que el usuario tenga permiso para crear logs/.
- No ejecutar como root salvo una necesidad explícita y comprendida; hacerlo amplía el impacto de comandos y operaciones destructivas.
- Mantener el ejecutable y sus directorios de datos bajo una ruta controlada por el usuario.
- Para distribuir binarios, compilar en una distribución compatible o empaquetar dependencias; GTK3 debe existir en el sistema destino.
- Antes de actualizar el código, conservar respaldos externos de los datos que se utilizarán en demostraciones.

11. Verificación y pruebas

11.1 Evidencia de revisión

Durante la elaboración de esta documentación se revisaron todos los módulos y cabeceras del proyecto. El objetivo CLI se compiló correctamente con gcc, -Wall, -Wextra y -std=gnu11, y se ejecutó una prueba de inicio/salida. La GUI requiere encabezados GTK3 durante la compilación; el proyecto incluye la configuración pkg-config correspondiente y existe evidencia de ejecución en el entorno Ubuntu del equipo.

11.2 Casos de prueba recomendados

ID	Procedimiento	Resultado esperado
CP-BLD-01	Ejecutar make cli desde un árbol limpio.	Se genera admin sin errores ni advertencias nuevas.
CP-GUI-01	Instalar GTK3 y ejecutar make gui.	Se genera admin-gui y abre una ventana con cuatro pestañas.
CP-PRO-01	Listar y buscar un proceso conocido.	El proceso aparece con PID, PPID, estado y RSS.
CP-PRO-02	Suspender y reanudar un proceso de prueba.	El estado cambia y el proceso continúa tras SIGCONT.
CP-ARC-01	Crear una carpeta desde GUI y también desde el explorador externo.	La carpeta aparece automáticamente en el directorio monitoreado.
CP-ARC-02	Copiar una carpeta con dos niveles y un enlace simbólico.	La estructura y el enlace se reproducen; el origen permanece intacto.
CP-ARC-03	Intentar copiar una carpeta dentro de sí misma.	La operación falla con EINVAL y mensaje explicativo.
CP-ARC-04	Buscar desde / un patrón conocido.	La GUI no se congela; se muestran rutas y el límite se respeta.

ID	Procedimiento	Resultado esperado
CP-CMD-01	Ejecutar: sh -c "echo ok; echo error &2; exit 3".	stdout=ok, stderr=error y código=3.
CP-RES-01	Crear respaldo, modificar un archivo y crear otra versión.	La primera versión es completa y la segunda contiene el archivo modificado.
CP-RES-02	Restaurar una versión incremental en carpeta vacía.	Se documenta que solo reaparecen los archivos presentes en esa versión.
CP-BAS-01	Analizar tests/prueba_bash_analyzer.sh .	Se detectan variables, for, while, if y función.
CP-SEG-01	Intentar finalizar un proceso ajeno sin permisos.	La operación falla y la aplicación continúa estable.
CP-MEM-01	Repetir listados y búsquedas observando consumo.	Las listas se liberan y no existe crecimiento sostenido significativo.

11.3 Pruebas automatizadas futuras

- Pruebas unitarias de archivos con directorios temporales y enlaces simbólicos.
- Pruebas de comandos con grandes volúmenes simultáneos en stdout y stderr.
- Pruebas de recuperación de respaldos completos e incrementales encadenados.
- Análisis de memoria con Valgrind o AddressSanitizer.
- Integración continua que compile CLI y GUI en Ubuntu con GTK3.
- Pruebas de seguridad para rutas maliciosas, permisos, comandos y enlaces simbólicos.

12. Seguridad, riesgos y limitaciones conocidas

12.1 Modelo de seguridad actual

ADMIN no implementa usuarios propios: hereda la identidad, permisos y límites del proceso que lo ejecuta. Esto simplifica el diseño, pero implica que cualquier comando o acción permitida por el sistema también puede ejecutarse desde la herramienta. El control principal es operar con una cuenta sin privilegios y confirmar las acciones destructivas.

12.2 Riesgos y limitaciones

ID	Nivel	Descripción	Tratamiento recomendado
R-01	Crítico	La ejecución usa /bin/sh -c sin lista blanca ni aislamiento.	Ejecutar solo comandos confiables; añadir modo seguro, perfiles y confirmación para comandos peligrosos.

ID	Nivel	Descripción	Tratamiento recomendado
R-02	Alto	La cola de descargas no descarga; únicamente simula progreso.	Integrar libcurl y comprobar códigos HTTP, TLS, tamaño, destino y hash.
R-03	Alto	Restaurar una versión incremental aislada no reconstruye el estado completo.	Crear snapshots completos, hard links o encadenar automáticamente todas las versiones necesarias.
R-04	Alto	stdout se lee completamente antes de stderr; una salida masiva en ambos pipes puede bloquear al hijo.	Usar poll/select/epoll o dos hilos para consumir ambos flujos concurrentemente.
R-05	Medio	Finalizar/suspender procesos no solicita confirmación en GUI.	Añadir diálogo con PID, nombre y señal antes de enviar kill().
R-06	Medio	Mover directorios entre sistemas de archivos no tiene fallback recursivo.	Aplicar copia recursiva + eliminación segura con verificación.
R-07	Medio	La copia no conserva propietario, tiempos, ACL, atributos extendidos ni contextos de seguridad.	Usar fchmod/fchown/futimens at y APIs de ACL/xattr según política.
R-08	Medio	El criterio incremental depende de mtime y reloj del sistema.	Usar manifiestos con tamaño/hash o metadatos persistentes por archivo.
R-09	Medio	No hay transacciones ni limpieza completa de destinos parciales tras un error.	Copiar a rutas temporales y realizar commit mediante rename().
R-10	Bajo	El monitoreo automático cubre solo el directorio actual, no todo el sistema.	Mantenerlo por eficiencia o añadir una vista de eventos con inotify configurable.
R-11	Bajo	Los logs se resuelven respecto del directorio de ejecución.	Usar XDG_STATE_HOME o una ruta configurable.
R-12	Bajo	El árbol de procesos está limitado a 2048 entradas.	Sustituir arreglos globales por memoria dinámica.

12.3 Recomendaciones operativas inmediatas

1. Ejecutar ADMIN con una cuenta de usuario normal.
2. Usar procesos y directorios de prueba durante demostraciones de finalizar, mover, eliminar y restaurar.
3. No introducir contraseñas ni secretos en comandos, porque el historial se guarda como texto plano.
4. Mantener una copia externa antes de probar respaldos o restauraciones.
5. No presentar la cola como descarga real; identificarla como simulación académica.
6. Cerrar la aplicación si una operación queda bloqueada y conservar logs para diagnóstico.

13. Mantenimiento y evolución

13.1 Estrategia de mantenimiento

Tipo	Acciones
Correctivo	Corregir errores reproducibles, añadir pruebas de regresión y mantener compatibilidad con la API pública.
Adaptativo	Actualizar compilación y llamadas obsoletas cuando cambien GTK/GLib o las distribuciones Linux.
Perfectivo	Mejorar experiencia de usuario, rendimiento, mensajes y cobertura de operaciones.
Preventivo	Reducir globals, validar memoria, documentar ownership y aplicar análisis estático.

13.2 Distribución recomendada del trabajo

Responsable	Módulos	Responsabilidades
Persona 1	archivos.c/.h, gui_archivos.c, comandos.c/.h, gui_comandos.c	Explorador, búsqueda, operaciones, ejecución e historial.
Persona 2	procesos.c/.h, gui_procesos.c, respaldos.c/.h, bash_analyzer.c/.h, gui_respaldos.c, main*, Makefile	Procesos, respaldos, análisis, integración y construcción.
Ambos	README, documentación, pruebas y revisión de Pull Requests	Pruebas cruzadas, trazabilidad y control de calidad.

13.3 Convenciones de contribución

```
git switch -c feature/<modulo>-<cambio>
make clean && make cli    # mínimo antes del commit
git add <archivos_del_modulo>
git commit -m "feat(archivos): agregar <funcionalidad>"
git push -u origin feature/<modulo>-<cambio>
```

- No mezclar cambios de módulos no relacionados en un mismo commit.
- Mantener mensajes feat, fix, docs, test, refactor, build o chore.
- Evitar commits de binarios admin, admin-gui, obj/ y logs.
- Requerir revisión de la otra persona antes de fusionar a main.
- Actualizar requisitos, riesgos y pruebas cuando cambie el comportamiento observable.

13.4 Hoja de ruta priorizada

Prioridad	Área	Resultado esperado
1	Seguridad y estabilidad	Lectura concurrente de pipes, confirmación de señales, validaciones de rutas y pruebas de memoria.
2	Respaldos confiables	Modelo de snapshots o cadena incremental reconstruible, manifiestos con hash y restauración verificable.
3	Descargas reales	libcurl, cancelación, progreso real, directorio destino y reintentos.
4	Pruebas automatizadas	Framework de pruebas C, directorios temporales e integración continua.
5	Empaquetado	Instalación en /usr/local o paquete .deb, archivos desktop e icono.
6	Observabilidad	Ruta XDG para logs, niveles de log, rotación y diagnóstico visible.

14. Conclusiones técnicas

ADMIN presenta una base modular sólida para un proyecto académico de Programación de Sistemas: combina lectura de /proc, señales, creación de procesos, pipes, operaciones de archivos, concurrencia con GLib y una interfaz GTK3. La separación entre presentación y núcleo permite demostrar los mismos conceptos desde terminal y escritorio sin duplicar la lógica principal.

Las funciones de procesos, exploración de archivos, captura de comandos y análisis Bash están claramente delimitadas y utilizan APIs propias de Linux de manera comprensible. La búsqueda en segundo plano y el monitoreo del directorio actual mejoran la respuesta de la GUI, mientras que la copia recursiva amplía el explorador a carpetas completas.

Para alcanzar un nivel de producción se deben priorizar seguridad de comandos, captura concurrente de salidas, respaldos restaurables como estado completo, operaciones atómicas y pruebas automatizadas. La cola de descargas debe mantenerse identificada como simulación hasta integrar una transferencia real.

Valor académico — El proyecto evidencia integración de procesos, señales, IPC mediante pipes, sistema de archivos, GTK3 y diseño modular en C, cumpliendo el objetivo de reunir conceptos centrales de administración y programación de sistemas.

15. Referencias

- [1] [ISO. ISO/IEC/IEEE 29148:2018 — Requirements engineering.](#)
- [2] [ISO. ISO/IEC 25010:2023 — Product quality model.](#)
- [3] [GTK Project. GtkApplication — GTK 3 API Reference.](#)
- [4] [GTK Project. GtkNotebook — GTK 3 API Reference.](#)
- [5] [GTK Project. GtkListStore and GtkTreeView — GTK 3 API Reference.](#)
- [6] [GTK Project. GFileMonitor — GIO API Reference.](#)
- [7] [Linux Kernel Documentation. The /proc Filesystem.](#)
- [8] [The Open Group. POSIX System Interfaces.](#)
- [9] [GNU Project. GNU Make Manual — Makefile Contents.](#)
- [10] Proyecto ADMIN. Código fuente, README, Makefile y documentación interna revisados el 21/07/2026.
- [11] Documento SRS proporcionado por el usuario como referencia de estructura.

Apéndice A. Estructura del repositorio

```
admin_proyecto/
├── Makefile
├── README.md
├── include/
│   ├── procesos.h
│   ├── archivos.h
│   ├── comandos.h
│   ├── respaldos.h
│   ├── bash_analyzer.h
│   ├── utils.h
│   └── gui_common.h
├── src/
│   ├── main.c
│   ├── main_gui.c
│   ├── procesos.c
│   ├── archivos.c
│   ├── comandos.c
│   ├── respaldos.c
│   ├── bash_analyzer.c
│   ├── utils.c
│   └── gui/
│       ├── gui_procesos.c
│       ├── gui_archivos.c
│       ├── gui_comandos.c
│       ├── gui_respaldos.c
│       └── gui_common.c
├── tests/prueba_bash_analyzer.sh
├── docs/
├── logs/
└── respaldos_data/
```

A.1 Responsabilidad de directorios

Ruta	Contenido
include/	Contratos públicos, estructuras y prototipos.
src/	Implementación de lógica núcleo y entradas CLI/GUI.
src/gui/	Callbacks y construcción de widgets por pestaña.
tests/	Entradas o scripts de prueba.
docs/	Manual y documentación técnica.
logs/	Historial y eventos generados en ejecución.
respaldos_data/	Ubicación sugerida para versiones de respaldo.

Ruta	Contenido
obj/	Objetos creados durante la compilación; no se versiona.

Apéndice B. Matriz de trazabilidad

Requisito(s)	Componentes de implementación	Prueba asociada
RF-PRO-01 a RF-PRO-06	src/procesos.c, include/procesos.h, src/gui/gui_procesos.c	CP-PRO-01, CP-PRO-02
RF-ARC-01 a RF-ARC-10	src/archivos.c, include/archivos.h, src/gui/gui_archivos.c	CP-ARC-01 a CP-ARC-04
RF-CMD-01 a RF-CMD-04	src/comandos.c, include/comandos.h, src/gui/gui_comandos.c	CP-CMD-01
RF-RES-01 a RF-RES-04	src/respaldos.c, include/respaldos.h, src/gui/gui_respaldos.c	CP-RES-01, CP-RES-02
RF-BAS-01 a RF-BAS-02	src/bash_analyzer.c, include/bash_analyzer.h	CP-BAS-01
RF-COL-01 a RF-COL-03	src/respaldos.c, src/gui/gui_respaldos.c	Prueba manual de cola
RF-UI-01 a RF-UI-02	src/main.c, src/main_gui.c, src/gui/*.c, Makefile	CP-BLD-01, CP-GUI-01
RNF-REN-01	src/gui/gui_archivos.c (GThread + g_idle_add)	CP-ARC-04
RNF-SEG-01 a RNF-SEG-02	Permisos del SO y diálogos GUI	CP-SEG-01
RNF-MAN-01 a RNF-MAN-02	Estructura modular y Makefile	Revisión estática + compilación

Apéndice C. Glosario

Término	Definición
ACL	Lista de control de acceso aplicada a un archivo o recurso.
CLI	Interfaz de línea de comandos.
CPU tick	Unidad contable utilizada por el kernel para tiempos de CPU.
exec	Familia de funciones que reemplaza la imagen del proceso actual.
fork	Función que crea un proceso hijo.
GFileMonitor	Objeto GIO que notifica cambios en un archivo o directorio.
GTK3	Toolkit gráfico utilizado por la interfaz de escritorio.
mtime	Fecha de última modificación de un archivo.
pipe	Canal unidireccional de comunicación entre descriptores.
POSIX	Conjunto de interfaces y comportamientos estandarizados para sistemas tipo Unix.
/proc	Pseudo-sistema de archivos que expone información del kernel y procesos.
PPID	Identificador del proceso padre.
RSS	Memoria física residente aproximada de un proceso.
SIGCONT	Señal para continuar un proceso detenido.
SIGSTOP	Señal no interceptable para detener un proceso.
SIGTERM	Señal de terminación que permite limpieza controlada.
stdout / stderr	Flujos estándar de salida normal y salida de error.
Snapshot	Representación de un estado completo en un momento determinado.
XDG	Convenciones de directorios de usuario en escritorios Linux.

— Fin del documento —